



RAPPORT DE PROJET Big data

Mini projet : Plateforme d'automatisation de transport **TaaSim**

Filière : ID2

Fait par : Kaoutar El wahid & Amine Al guardoum & Abdelkader Anmira

Encadré par : M. Marouani

1. Introduction, contexte et problématique :

Casablanca est une grande ville avec un grand nombre d'habitants. Les habitants perdent du temps à la recherche d'un taxi, qui peut passer au hasard ou après un long temps d'attente, alors que les taxis circulent eux aussi de manière aléatoire en cherchant des demandes. Notre plateforme vise à résoudre ce problème en centralisant les offres des chauffeurs et les demandes des clients, en mettant en œuvre une infrastructure Big Data capable de faire correspondre les demandes et les offres, ainsi que d'analyser les performances du réseau et de prédire la demande à court terme.

2. Méthodologie et jeu de données :

Parmi les difficultés que nous avons rencontrées avant de travailler sur ce projet, il y avait l'absence de données géospatiales. Nous avons utilisé deux datasets : Porto pour les trajectoires des taxis (1,7 million de trajets détaillés sur 12 mois) et le jeu de données massif **NYC TLC Trip Records** (30 millions de trajets au format Parquet pour la couche batch).

Pour adapter ces données au contexte de Casablanca, nous avons procédé à un géomapping : nous avons transformé linéairement les coordonnées GPS de Porto afin de les faire correspondre à la zone géographique de Casablanca (définie par une latitude entre 33.500 et 33.650 et une longitude entre -7.700 et -7.500). Nous avons aussi réassigné les 22 zones d'activité de Porto aux 16 arrondissements officiels de Casablanca, comme Anfa, Maarif, Hay Hassani ou

Sidi Moumen. Le streaming temps réel est émulé par un script Python rejouant les trajets à une vitesse accélérée (10x) dans Kafka.

3. Architecture globale (Framework Kappa)

Pour ne pas écrire le code deux fois et assurer la cohérence entre le batch et le streaming, nous avons utilisé une architecture Kappa.

Dans ce modèle, Apache Kafka sert de journal d'événements unifié et de système d'enregistrement de référence (rétention de 7 jours). Le traitement s'applique dans un seul moteur de flux, Apache Flink, tandis qu'Apache Spark intervient uniquement de manière asynchrone pour l'ETL analytique lourd et l'apprentissage automatique hors ligne.

4. Ingestion et traitement temps réel (Kafka & Flink)

Le cluster Kafka fonctionne en mode KRaft. Il gère notamment les topics **raw.gps** et **raw.trips**, qui représentent respectivement les positions GPS et les demandes de trajets.

Nous avons mis en place un connecteur S3 Sink (Kafka Connect) qui archive automatiquement ces flux de données brutes dans un espace de stockage MinIO, dans le dossier raw/kafka-archive/.

Le traitement des données en temps réel est organisé en trois applications Flink distinctes :

- **Job 1 (GPS Normalizer)** : il nettoie les données en supprimant les coordonnées GPS erronées, applique une tolérance de retard de 3 minutes pour gérer les éventuels délais, anonymise les positions en les regroupant au centre de chaque arrondissement, puis stocke ces informations dans la base `vehicle_positions` sur Cassandra.
- **Job 2 (Demand Aggregator)** : cette application regroupe les données GPS et les demandes de trajets sur des périodes de 30 secondes pour calculer le nombre de taxis actifs et les réservations en cours. Elle calcule ensuite le ratio entre l'offre et la demande, avant de sauvegarder ces résultats dans `demand_zones` sur Cassandra.
- **Job 3 (Trip Matcher)** : ce dernier garde une trace des taxis disponibles dans RocksDB et associe chaque demande à un taxi libre. Si aucun taxi n'est immédiatement disponible, il attend 5 secondes avant d'élargir la recherche aux arrondissements voisins, puis enregistre les correspondances trouvées.

5. Modélisation de Données (Cassandra NoSQL)

En respectant les standards et les principes de NoSQL, , voilà le schéma Cassandra qui assure l'accès en lecture/écriture sous la milliseconde :

La table **vehicle_positions** stocke les positions en temps réel des taxis actifs, en enregistrant leur localisation, leur vitesse et leur statut pour chaque zone et ville. Les données sont organisées afin de permettre un accès rapide aux positions les plus récentes.

La table **trips** est utilisée pour le suivi des trajets, en conservant les informations relatives aux passagers, aux zones de départ et d'arrivée, au statut du trajet, au taxi attribué et au temps estimé d'arrivée. Les trajets sont regroupés par ville et par jour pour faciliter leur consultation.

6. Traitement Batch & Machine Learning (Spark)

Maintenant, on discute le pipeline de spark ETL, les jobs de Pyspark popularisent chaque jour les archives de minIO, on fait plusieurs transformations comme le geospatial en utilisant h3 qui a été créé par UBER, pour éviter de calculer le ping GPS et clients un par un, car on s'intéresse dans notre modèle de demande par zone (h3 transforme ces coordonnées depuis coordonnées long/lat en plusieurs pièces groupées) et sauvegarde les données nettoyées sous format Parquet compressé en utilisant Snappy partitionné par mois, en utilisant spark on calcule aussi les KPIs dashboards (durée moyenne, zone de déficit des offres, heures...) tout cela persisté au Cassandra

Pour la tâche de prévision nous avons entraîné un modèle GBRegressor dans Spark MLlib, il est le plus efficace en fonction de performance et vitesse au même temps, mais avant on a fait bien sûr un feature engineering pour exploiter les variables temporelles (demande hier, demande de semaine dernière...) et variable de météo (via API Open Meteo) le modèle a été validé sur les derniers 2 mois et surpasse la baseline naïve 7 jours, il est servi via FastAPI

7. API & Hardening Sécurité

Comme déjà dit aux étapes précédentes, FastAPI offre des endpoints asynchrones sécurisés pour la réservation et la demande des trips (POST /trips), le statut du trajet, la liste des taxis par zone et la préservation de la demande en utilisant le modèle ML construit par GBRegressor. La sécurité est garantie avec l'utilisation de JWT, qui fait la distinction entre les rôles (Rider, Admin/Rider, Client) et la sécurisation des endpoints. Restructuration et modularisation de l'API : le script API monolithique a été refactorisé en une architecture évolutive et modulaire. Services (src/api/services/) : séparation des connexions aux services externes. Routes (src/api/routes/) : division des fonctionnalités en modules distincts. Core (src/api/core/) : configurations, journalisation et dépendances. Avec l'intégration de Docker, ajout d'un Dockerfile.api spécifique pour conteneuriser proprement l'application FastAPI, ainsi que d'un fichier .dockerignore afin de garantir que le contexte de construction reste léger

8. Métriques de Performance (SLAs) & Conclusion

Les tests d'intégration montrent que le système est performant et respecte les SLA. Les temps de réponse sont rapides (1,2 s pour le matching et 120 ms pour l'API ML) et les mises à jour sont régulières. Le traitement batch prend environ 2 min 14 s et tous les résultats sont conformes. Le système est aussi robuste, avec une reprise rapide de 12 secondes sans perte de données après panne.